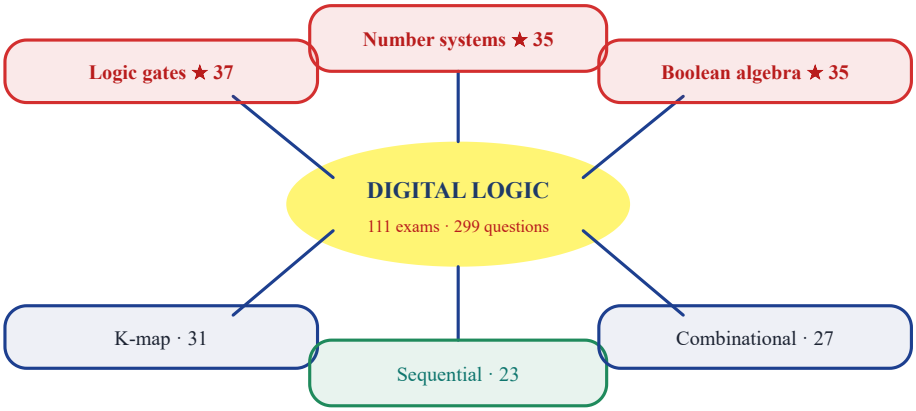
Digital Logic Design

every DLD question asked in BCS · NTRCA · BPSC · bank · power-sector · BUET-set papers, solved in 18 sheets



↑ the number is how many different exams asked that topic (red = most asked)

How to read a question card

> **Q:** The question, as printed in the paper (typos fixed).
> ✎ **Asked in:** the exams it was taken from, e.g. 46th BCS ICT · Combined Bank SO(IT) 2026

"Also asked" = the same pattern with other numbers. Green = checked answer.

Most asked

① latch vs flip-flop · 5
② K-map Σ(0,3,5,7,8,10–15) · 4
③ NAND is universal · 3
④ K-map Σ(2,8,9,11,13,15) · 3
⑤ FA from 2 HAs · 3-to-8 decoder
⑥ 2's complement subtraction

Syllabus coverage

"Digital logic: **Boolean algebra**, **De Morgan's Theorems**, logic gates and their truth tables, **canonical forms**, combinational logic circuits, **minimization techniques**", plus codes, flip-flops and counters.

> **Legend:** **yellow** = key term · **red circle** = must-memorise · **purple box** = memory trick · **red box** = exam trap · **green** = checked answer

Contents

✓ Logic gates I, II …… p. 2–3
✓ Number systems: bases, sums …… p. 4–5
✓ 2's complement · codes, parity … p. 6–7
✓ Boolean algebra: laws, simplify …… p. 8–9
✓ Truth tables, word problems …… p. 10
✓ K-map I: SOP · K-map II: POS, NAND p. 11–12
✓ Adders and subtractors …… p. 13
✓ Decoders, encoders, MUX …… p. 14
✓ Latches and flip-flops …… p. 15
✓ Counters and FSMs …… p. 16
✓ Night-before sheet …… p. 17
✓ Practice MCQs + written …… p. 18

1 Basic gates and truth tables

Q1: What is a logic gate? How many basic gates are there? Describe them with truth tables. / What is a truth table? Give an example.

✎ **Asked in:** BPSC ANE 2020 · 17th NTRCA School (2023) · 15th NTRCA Computer Operation (2019) · BPSC ADA 2022

A **logic gate** is an electronic circuit that produces one binary output from one or more binary inputs by a fixed rule. A **truth table** lists the output for all 2^n input rows. **3 basic gates**: AND, OR, NOT; NAND, NOR universal; XOR, XNOR derived.

A B	AND AB	OR A+B	NAND (AB)′	NOR (A+B)′	XOR A⊕B	XNOR (A⊕B)′	NOT A′
0 0	0	0	1	1	0	1	A=0 → 1
0 1	0	1	1	0	1	0	A=1 → 0
1 0	0	1	1	0	1	0	
1 1	1	1	0	0	0	1	



Q2: EX-OR gate with truth table. / X-OR and X-NOR truth table and diagram.

✎ **Asked in:** 17th NTRCA CSE (2023) · BCPCL JI 2019 · Cadet College Lecturer(ICT) 2025

XOR is 1 when the inputs **differ**: A⊕B = A′B + AB′. XNOR is 1 when they are **equal**: (A⊕B)′ = AB + A′B′ (the **coincidence** gate). With n inputs XOR = 1 for an **odd** number of 1s → parity generator.

Q3: Compare TTL and CMOS in speed, noise and power. / Characteristics of digital ICs.

✎ **Asked in:** Hydrocarbon Unit AD(ICT) 2025 · Combined Bank AME/AHE 2023

TTL (74xx): ~10 ns, noise margin ≈ 0.4 V, mW per gate even when idle. CMOS (74HC/4000): similar speed, noise margin ≈ 30 % of V_{DD}, **static power ≈ 0** (power only while switching). IC characteristics: fan-in, **fan-out**, propagation delay, power dissipation, noise margin.

MCQ as asked	answer	asked in
The "anticoincidence" gate	**XOR**	49th BCS ICT (2025)
Output LOW when any one input is HIGH	**NOR**	BPSC AP 2022
Which circuit stores 1 bit?	**flip-flop**	DoICT ANE 2019
10101111 OR 00110011	**10111111**	DoICT ANE 2019

Memory trick: **"AND needs All, OR needs One, XOR needs Odd"**. The N-version is the same column flipped.

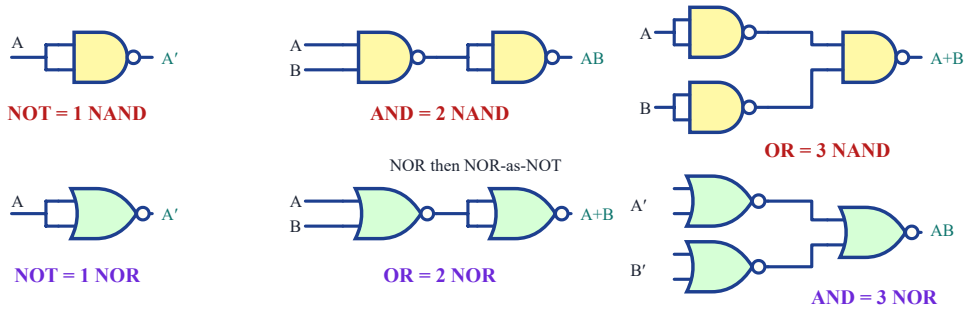
Exam trap: A register stores **n** bits and is *made of* flip-flops; the 1-bit store is the flip-flop (or latch).

2 NAND and NOR are universal

Q4: What is a universal gate? Prove that the NAND gate is a universal gate. / Why are NAND and NOR called universal gates? / Prove the universality of NOR.

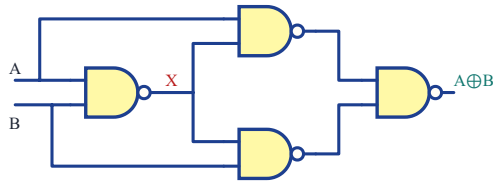
✎ **Asked in:** Combined Bank AP 2026 · SPCBL AME 2021 · Bank (name cut off) AP · 18th NTRCA School (2024) · 15th NTRCA Computer Operation (2019) · 16th NTRCA CSE (2019) · 13th NTRCA CSE (2016) · NPCBL JAM 2022 · BRICM AME 2024 · 46th BCS CSE

A **universal gate** alone builds every Boolean function, since it makes the complete set {NOT, AND, OR}: **proof** = draw those three. Advantages: one IC type, cheaper stock, fastest/smallest gate in TTL and CMOS.



Q5: Implement XOR using at most 4 NAND gates (with figure).

✎ **Asked in:** PGCB AE(CSE) 2021 · Combined Bank Officer(IT) 2026 · 46th BCS CSE



X = (AB)′; out = ((A·X)′ · (B·X)′)′ = AX + BX = (A+B)(AB)′ = **A⊕B** with 4 NAND. XNOR = this + one NAND as NOT → 5 ✎ **Asked in:** 15th NTRCA CSE

Q6: What are universal gates? Implement F = AB + CD + E using NAND gates only.

✎ **Asked in:** 45th BCS ICT (2024)

Double-invert, De Morgan: **F = ((AB)′·(CD)′·E′)′** → NAND(A,B), NAND(C,D), E′ = NAND(E,E), then one 3-input NAND.

→ 3 two-input NAND + 1 three-input NAND

Q7: OR (F1 = A+B) and AND (F2 = AB) with the minimum number of NAND and of NOR gates.

✎ **Asked in:** PGCB AE(CSE) 2024 · BUET M.Sc. May 2019

→ AB: **2 NAND** / 3 NOR · A+B: 3 NAND / **2 NOR**

Also asked: NAND from NOR (4 NOR: AND from 3, then 1 as NOT) ✎ BMA AE 2021 · OR from NAND ✎ DESCO SAE 2022 · AND, OR, NOT from NAND and NOR ✎ BPSC AP 2021 · Intl Crimes Tribunal Hardware Technician 2024

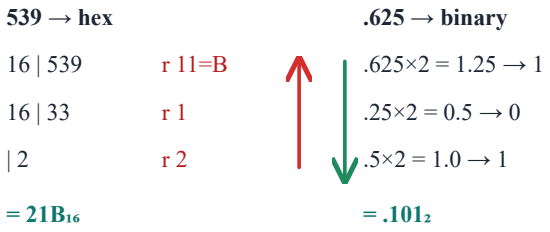
· 17th NTRCA School (2023) · a 3-input universal gate: symbol + 8-row table (NAND3 = 0 only for 111) ✎ BPSC AP 2021 · (A+B)(C+D) with NAND: NAND(A′,B′)·NAND(C′,D′), then NAND + NOT ✎ Fisheries & Livestock Instructor(CS) 2023 · F = A + (B′+C)(D′+BE′) and F = ((A+B)+CD)E with NAND only ✎ NWPGCL JAM(ICT) 2022 · Universal gate MCQ → **NAND** ✎ BB AD(ICT) 2025

> **Memory trick:** **NAND loves AND** (AND = 2 NAND, OR = 3); **NOR loves OR** (OR = 2 NOR, AND = 3).

3 Conversions between bases

The **base (radix)** is the number of digit symbols; a digit's value = digit × base^{position} (**positional** system). Roman numerals are **non-positional**. ✎ **Asked in:** 17th NTRCA School (2023)

Decimal → base b: integer by repeated **division** (remainders bottom-up), fraction by repeated **multiplication** (top-down). Bin ↔ oct/hex: group **3 / 4 bits**.



> **Q8:** Write the decimal, octal and binary form of $(E)_{16}$. Write the next four numbers in the hexadecimal sequence 6FC, 6FE, __, __, __, __.
> ✎ **Asked in:** 45th BCS CSE (2024)

→ $(E)_{16}$ = **$(14)_{10}$ = $(16)_8$ = $(1110)_2$** · the sequence steps by 2: 6FE + 2 = **700, 702, 704, 706** (FE + 2 carries)

> **Exam trap:** E = **14**, not 15 (F = 15); a printed solution used 15 and got every form wrong.

> **Q9:** Convert decimal 250.5 to base 3 and base 7, keeping three digits after the point.
> ✎ **Asked in:** 47th BCS ICT

250 = 3^5 + 2·3 + 1, 0.5×3 = 1.5 → 1 (repeats) → **100021.111_3**
250 = 5·49 + 5, 0.5×7 = 3.5 → 3 → **505.333_7**

Asked conversion	checked answer	asked in
$(39.B8)_{16}$ → decimal	57 + 184/256 = **57.71875**	45th BCS ICT (2024)
$(423)_{10}$ → octal · $(3000)_{10}$ → hex	**647_8** · **$BB8_{16}$**	BPDB AE(CSE) 2024
$(CDAB)_{16}$ → octal	1100 1101 1010 1011 → **146653_8**	BTCL JAM 2024
$(1741)_{10}$ → hex · $(12345)_{10}$ → octal	**$6CD_{16}$** · **30071_8**	DESCO AE(IT) 2022 · BB AP 2019
$(10010.101)_2$ → dec · $(543)_{10}$ → hex	**18.625** · **$21F_{16}$**	Combined Bank Officer(IT) 2020
$(2345)_{10}$ → hex · $(ABCD)_{16}$ → octal	**929_{16}** · **125715_8**	ICT Ministry AP 2017
$(651)_8$ → decimal, hex	**425** · **$1A9_{16}$**	BPSC AME 2021

Memory trick: Hex digit = 4 bits, octal digit = 3 bits: **"8-4-2-1"** for a nibble. Hex → octal: pass through binary, regroup by 3 from the point.

4 Fractions and sums across bases

Asked conversion	checked answer	asked in
$(1AC)_{16}$ → decimal · $(539)_{10}$ → hex, oct	**428 · $21B_{16}$, 1033_8**	DPDC JAM(ICT) 2020 · BCPCL JI 2019
$(1111001101011)_2$ → octal, hex	**17153_8 · $1E6B_{16}$**	NWPGCL JAM 2020
$(90 - 75)_{10}$ → binary, hex	15 = **1111_2 = F_{16}**	17th NTRCA CSE (2023)
$(73.75)_{10}$ → binary, hex	**1001001.11_2 · $49.C_{16}$**	15th NTRCA Computer Operation (2019)
$(90.625)_{10}$ → bin · $(AB.C)_{16}$ → dec	**1011010.101_2 · 171.75**	16th NTRCA CSE (2019)
$(53.62)_{10}$ → octal	**$65.4753\ldots_8$** (recurs)	17th NTRCA School (2023)
$(10110.11)_2$ → hex · $(AB.4C)_{16}$ → bin	**$16.C_{16}$ · 10101011.010011_2**	13th NTRCA CSE (2016)
MCQ $(101.101)_2$ = ? · next after 77_8 = ?	**5.625 · 100_8**	49th BCS ICT (2025) · BPSC AME 2022

Q10: Add $(9D.AB6)_{16}$ and $(306.51)_{10}$ and express the result in binary. Which number system could (110101) belong to?
✎ **Asked in:** 18th NTRCA School (2024)

To decimal: $9D.AB6_{16}$ = 157 + 2742/4096 = 157.6694 → + 306.51 = **464.1794…** → 464 = 111010000, fraction × 2 repeatedly.

→ ≈ **111010000.0010110111_2** (the fraction does not terminate: say "to 10 places") · (110101) uses only 0 and 1, so it is a valid numeral in **every base ≥ 2**: binary (= 53), octal, decimal, hex…

Exam trap: Some copies print this as $(9D.AB)_{16}$ + $(336.51)_8$ = 380.3086 → 101111100.0100111100_2. The printed paper has **$(9D{\cdot}AB6)_{16}$** and **$(306{\cdot}51)_{10}$**; the "·" is the decimal point. Copy the base subscripts carefully before you start.

Q11: Convert $(16A.6)_{16}$ and $(72.3)_8$ to binary and show their sum in hexadecimal.
✎ **Asked in:** 46th BCS CSE

$16A.6_{16}$ = 1 0110 1010.0110 · 72.3_8 = 111 010.011 → line up the points and add in binary:

→ 110100100.11_2 = **$1A4.C_{16}$** (= 362.375 + 58.375 = 420.75)

Q12: Change $(3D.4C)_{16}$ and $(514.6)_8$ to binary, add them and show the sum in hexadecimal.
✎ **Asked in:** 16th NTRCA CSE (2019)

→ 61.296875 + 332.75 = 394.046875 = 110001010.000011_2 = **$18A.0C_{16}$**

Also asked: sums

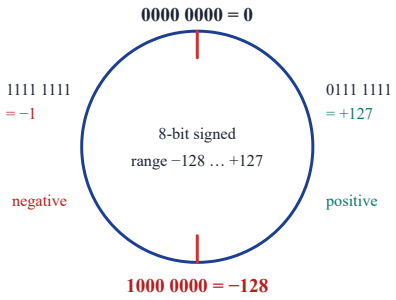
- $(BF)_{16}$ + $(55)_8$, result in hex → 191 + 45 = 236 = **EC_{16}** ✎ 17th NTRCA School (2023)

- $(456)_8 + (2D)_{16}$ in binary, result in decimal → 100101110 + 101101 = **101011011_2 = 347** ✎ 15th NTRCA CSE

5 2's complement arithmetic

> **Q13:** What is the 2's complement method? Subtract 27 from 55 using 2's complement.
> ✎ **Asked in:** BTCL JAM 2024

2's complement of an n-bit number = invert every bit (1's complement) **+ 1** ($= 2^n - x$). To compute A − B, add A and the 2's complement of B. A carry out of the MSB is **discarded**; if the MSB of the result is 1 the answer is negative, and its size is the 2's complement of the result.




```
 55 = 0011 0111
-27 → 1110 0101   (27 = 0001 1011)
    1 0001 1100   drop the carry
      0001 1100 = +28 ✓
```

Also asked (8 bits)	result	asked in
$(5)_{10} - (7)_{10}$	11111110 = **−2**	18th NTRCA School-2 (2024)
$(-13)_{10} - (9)_{10}$	11101010 = **−22**	17th NTRCA School (2023)
$(32)_{10} - (18)_{10}$	00001110 = **+14**	15th NTRCA Computer Operation (2019)
$(-15)_{10} - (+11)_{10}$	11100110 = **−26**	18th NTRCA School (2024)
$(108)_8 - (78)_{16}$	11110100 = **−12**	17th NTRCA CSE (2023)
−25 in 8-bit 2's complement	**11100111**	Hydrocarbon Unit AD(ICT) 2025
2's complement of $(+55)_{10}$	**11001001** (= −55)	45th BCS CSE (2024)

> **Exam trap:** "$(108)_8$" cannot exist: **8 is not an octal digit**. Write that first, state your reading ($(108)_{10}$ here), then solve. Examiners give the method marks.

MCQs as asked

- 8-bit N: which operation leaves its lower 4 bits unchanged — XOR / NAND / AND / OR with 0Fh? → **AND (logical product)**; the printed key (NAND) inverts them ✎ Combined Bank AHE/AE(IT) 2021
- 1100101001 − 1100110110 (binary) → 809 − 822 = **-1101_2 (−13)** ✎ 45th BCS ICT (2024)
- Method mainly used for negative numbers in computers → **2's complement** ✎ 49th BCS ICT (2025)
- 10110011_2 could be read as 179 / B3 / −77 / all → **all of these** ✎ 49th BCS ICT (2025)
- 8-bit value of 10000000 in 2's complement → **−128** ✎ DoICT AP 2020
- 2's complement of 00000000 → **00000000** ✎ BUET IICT April 2021
- 2's complement form of $(67)_{10}$ → **01000011** (−67 = 10111101) ✎ BB AD(ICT) 2025

Also asked: decimal → base 9, octal → decimal, octal → hex (same two methods) ✎ Cadet College Lecturer(ICT) 2025

Memory trick: **"Copy up to the first 1 from the right, then flip the rest"**: 0011 0111 → 1100 1001 in one pass.

6 Codes and parity

Q14: What is a code? Distinguish BCD and binary code. / Describe BCD, ASCII and EBCDIC codes. / Features of ASCII.

✎ **Asked in:** 18th NTRCA School (2024) · 17th NTRCA School (2023) · 15th NTRCA Computer Operation (2019)

	pure binary	BCD (8421)
$(39)_{10}$	100111	0011 1001
bits	fewest	4 per digit
arithmetic	simple	needs +6 fix
used in	CPU, memory	displays, meters

A **code** is a fixed bit pattern for each symbol. **ASCII**: **7 bits**, 128 symbols ('A' = 65, 'a' = 97, '0' = 48), 8-bit extended forms. **EBCDIC**: 8 bits, IBM mainframes. **Excess-3**: BCD + 3, self-complementing.

Q15: Explain the parity method for error detection. Write the bit strings of "Delta" in ASCII.

✎ **Asked in:** BB AP 2023

A **parity bit** makes the number of 1s even (even parity) or odd (odd parity). The receiver recounts: a mismatch = an odd number of bit errors (2 errors pass unseen).

char	ASCII	7 bits	even P
D	68	1000100	0
e	101	1100101	0
l	108	1101100	0
t	116	1110100	0
a	97	1100001	1

Q16: BCD addition: 00010011 + 00100110. / Excess-3 code of 1010?

✎ **Asked in:** NPCBL JAM 2022 · DPDC JAM(ICT) 2020

→ 13 + 26: 0001 0011 + 0010 0110 = **0011 1001** = 39 (no digit > 9, so no +6)

→ 1010 = $(10)_{10}$: digits 1, 0 → each + 3 → **0100 0011**

Q17: In which logic is 1 + 1 = 1, and in which is 1 + 1 = 10? Explain.

✎ **Asked in:** 17th NTRCA School (2023)

Boolean OR: 1 + 1 = 1 (true OR true is true). **Binary arithmetic**: 1 + 1 = 10 (sum 0, carry 1); 1 + 1 + 1 = **11**

✎ **Asked in:** BDCCL AM(Transmission) 2022

Also asked

- Even-parity bit for message 111 → **1** (makes four 1s) ✎ BUET IICT April 2021
- ASCII of 'C' · of 'b' → **67 · 98** ✎ BB AD(General) 2021 · DoICT AP 2020
- Bits in a BCD digit · format used to store numbers for display → **4 · BCD** ✎ BDCCL AM(Transmission) 2022 · BUET IICT April 2021
- In modulo-2 arithmetic, which two give the same result? → **addition and subtraction** (both XOR) ✎ 49th BCS ICT (2025)

- Base-3 (ternary) number system → **digits 0, 1, 2; $21_3 = 7$** ✎ 18th NTRCA School (2024)
- Why is binary logic used in digital systems? → **2 stable states: noise-proof, cheap** ✎ BWDB AP 2018
- 16 × 35 with only three 1-digit multiplications → 1·3, 6·5, (1+6)(3+5) → 3, 30, 56−33 = 23 → **560** ✎ BUET M.Sc. May 2016
- 32-bit DAC over 0–3.3 V: step size → 3.3 / ($2^{32} - 1$) ≈ **0.77 nV** (12-bit: 0.806 mV) ✎ Combined Bank Officer(IT) 2024

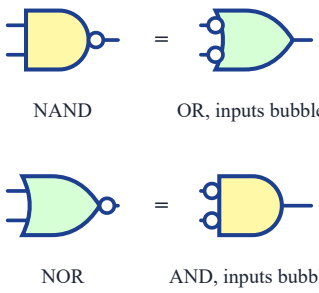
7 Laws, De Morgan and algebraic simplification

Q18: Define Boolean algebra. State its fundamental laws and show how they simplify logic expressions.
✎ **Asked in:** 47th BCS ICT

law	AND form	OR form	law	AND form	OR form
identity	A·1 = A	A + 0 = A	idempotent	A·A = A	A + A = A
null	A·0 = 0	**A + 1 = 1**	complement	A·A′ = 0	A + A′ = 1
absorption	A(A + B) = A	A + AB = A	redundancy	A(A′ + B) = AB	**A + A′B = A + B**
distributive	A(B + C) = AB + AC	A + BC = (A + B)(A + C)	consensus	**AB + A′C + BC = AB + A′C**	

Boolean algebra: the algebra of two values {0, 1} with AND, OR, NOT. Use the laws to cut literals, then draw fewer gates.

Q19: State and prove De Morgan's theorem (two and three variables) with a truth table. / De Morgan's duality formula with an example.
✎ **Asked in:** BPSC AP 2023 · 46th BCS ICT · 16th NTRCA CSE (2019) · DESCO SAE(Computer) 2026 · BCPCL JI 2019



break the bar, change the sign

1: (A + B)′ = A′B′ **2:** (AB)′ = A′ + B′; three variables: (A + B + C)′ = A′B′C′, (ABC)′ = A′ + B′ + C′. **Proof:** the 4 (or 8) rows of both sides agree. **Duality:** swap + ↔ · and 0 ↔ 1 and a true identity stays true.

A B	(A+B)′	A′B′	(AB)′	A′+B′
0 0	1	1	1	1
0 1	0	0	1	1
1 0	0	0	1	1
1 1	0	0	0	0

Q20: Prove: (i) $(A \oplus B)' = AB + A'B'$; (ii) if $F = X'Y + XYZ'$ then $F \cdot F' = 0$; (iii) $(A + B)' \cdot (A' + B)' = 0$ and $A + A'B + (AB)' = 1$.

✎ **Asked in:** 16th NTRCA CSE (2019) · 17th NTRCA School (2023) · 15th NTRCA Computer Operation (2019)

(i) $(A'B + AB')' = (A'B)'(AB')' = (A + B')(A' + B) = AA' + AB + A'B' + BB' =$ **$AB + A'B'$**. (ii) any $F \cdot F' = 0$ by the complement law. (iii) $A'B' \cdot AB' = 0$ $(A \cdot A')$; $A + A'B + A' + B' \supseteq A + A' =$ **1**.

8 Simplification and circuit ↔ truth table

Simplify (answers checked over every input)

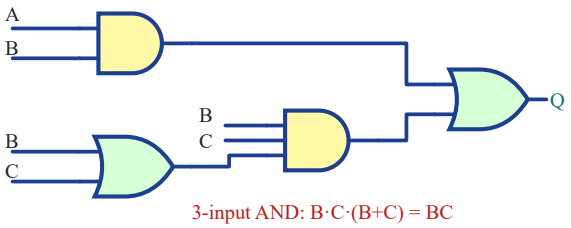
- $(x + y + xy)(x + z) = (x + y)(x + z) \rightarrow$ **$x + yz$** ✎ BPSC AME 2021
- $Y = AB + (A' + B)C = AB + A'C + BC \rightarrow$ **$AB + A'C$** (consensus drops BC) ✎ BPSC ADA 2022
- $xyz + x'yz + xy' = yz(x + x') + xy' \rightarrow$ **$yz + xy'$** ✎ BPSC AP 2021
- $yz + xz + xyz \rightarrow$ **$z(x + y)$** ✎ BAERA Engineer(CSE) 2023 · $AB + BC(B + C) \rightarrow$ **$B(A + C)$** ✎ 15th NTRCA CSE
- $F = A'B'C + A'BC + AB' = A'C(B' + B) + AB' \rightarrow$ **$A'C + AB'$** ✎ 45th BCS CSE (2024)
- $[(A + C')(B' + D)]' = A'C + BD' \rightarrow$ draw 2 AND + 1 OR ✎ 17th NTRCA School (2023) · $[x + y'(x + z)]' \rightarrow$ **$x'(y + z')$** ✎ 16th NTRCA CSE (2019)
- $A'B'C + AB'C' + ABC \rightarrow$ **$B'(A \oplus C) + ABC$** (no SOP term can be dropped) ✎ 17th NTRCA CSE (2023)
- $AB + (A(BC))(AC + (BC)') = AB + ABC \rightarrow$ **AB** ✎ NPCBL JAM 2022
- $ABCD + A'BD + AB'C'D \rightarrow$ **$A'BD + BCD + AB'C'D$** ✎ DPDC JAM(ICT) 2020 · $AB'C'D + ABCD + AB'D' \rightarrow$ **$ABCD + AB'C' + AB'D'$** ✎ APSCL AE(ICT) 2021
- $AB'C'D + A'B'D' + AB'CD$ in minimum literals $\rightarrow$ **$AB'D + A'B'D'$** ✎ SGFL AGE 2021
- $A'B'C' + B'CD' + A'BCD' + AB'C' \rightarrow$ **$B'C' + B'D' + A'CD'$** · $AB'CD' + A'B'C'D + A'BC'D + AB'C'D' \rightarrow$ **$AB'D' + A'C'D$** ✎ BPSC SAE 2021

MCQs: $(A + B + AB)(A + C) \rightarrow$ **$A + BC$** ✎ Combined Bank AHE/AE(IT) 2021 · $Y'(X + X'Y) \rightarrow$ **XY'** ✎ 49th BCS CSE (2025) · which is correct: $A + 1 =$ **1** ✎ BDCCL AM(Transmission) 2022 · minimum literals of $wx + wx' + xyz' + x'yz' + xy'z' + x'y'z' \rightarrow$ **$w + z'$** ✎ BUET M.Sc. April 2021

Memory trick: **"Break the bar, change the sign"** (De Morgan). **"$A + A'B$: the A' falls off"** (redundancy).

Q21: Draw the logic circuit of $Q = AB + BC(B + C)$ and find Q for input (A, B, C) = (1, 0, 1).

✎ **Asked in:** Combined Bank AME/AE(IT) 2024



3-input AND: $B \cdot C \cdot (B+C) = BC$

$BC(B + C) = BC$ (absorption), so **$Q = AB + BC = B(A + C)$**: one AND + one OR after simplifying. At (1, 0, 1): $B = 0 \rightarrow$ **$Q = 0$**.

- $Q = (A'B + BC(B + C)')'$ at (1, 0, 1) $\rightarrow$ inner $= 0 \rightarrow$ **$Q = 1$** ✎ Combined Bank AME/AE(IT) 2024
- $Q = C' + A'B + BC(B + C)$ at (0, 0, 1) $\rightarrow$ **$Q = 0$** ✎ Sonali Bank ADBA 2024 · $Q = C + AB + BC(B + C)$ at (0, 0, 1) (printed "0,0,10") $\rightarrow$ **$Q = 1$** ✎ Combined Bank ADA 2024
- Majority circuit (output 1 when more 1s than 0s) MCQ $\rightarrow$ **$AB + BC + CA$** ✎ BUET M.Sc. April 2021

- Circuit: AB and (A + B)′ combined, then inverted → F = (AB + (A + B)′)′ = **A ⊕ B**, output 1 for inputs **01 and 10** ✎ BAPEX AGM(ICT) 2023
- Traffic-light truth table (red, yellow, green) → write one SOP per lamp from its 1-rows, then simplify ✎ BAERA Engineer(CSE) November 2021

Also asked (figure only, not in the source): truth table from the given circuit ✎ Combined Bank ADA 2024 · DESCO AE(CSE) 2016 · minimise the given circuit / which circuit fits this table ✎ BUET M.Sc. October 2018

9 From words and expressions to truth tables

Q22: Determine the truth table of (A ⊕ B) ∨ (¬A) ∨ (¬C).
✎ **Asked in:** 45th BCS CSE (2024)

A B C	X = A⊕B	¬A	¬C	F
0 0 0	0	1	1	1
0 0 1	0	1	0	1
0 1 0	**1**	1	1	1
0 1 1	1	1	0	1
1 0 0	**1**	0	1	1
1 0 1	1	0	0	1
1 1 0	**0**	0	1	1
1 1 1	0	0	0	**0**

→ F = 0 only for A = B = C = 1, so **F = (ABC)′**

Exam trap: A printed solution has the X = A⊕B column wrong in the three bold rows (it still got F right). Examiners mark every column: fill X row by row, "1 when A and B differ".

Q23: Truth table for an alarm (output 1) when (motion is detected AND it is night) OR the panic button is pressed.
✎ **Asked in:** Combined Bank AP(SB & Janata) 2026

M = motion, N = night, P = panic → **Alarm = M·N + P**. Rows MNP = 000…111 give **0 1 0 1 0 1 1 1**: 1 whenever P = 1, and for M = N = 1.

Q24: Draw a truth table for Y = AB′CD′ + ABC′ + A′BD + AB′CD.
✎ **Asked in:** 38th BCS (2018)

Mark the rows each term covers: AB′CD′ → 1010 (10); ABC′ → 1100, 1101 (12, 13); A′BD → 0101, 0111 (5, 7); AB′CD → 1011 (11). Every other row of the 16 is 0.

→ Y = Σm(5, 7, 10, 11, 12, 13) = **A′BD + AB′C + ABC′** (a 4-variable K-map gives this)

Q25: Draw the logic circuit of X = A′BC + AB′C + ABC′ + ABC, and implement the simplified form with NAND and with NOR gates.
✎ **Asked in:** 13th NTRCA CSE (2016) · 18th NTRCA School (2024)

Q26: 3-bit input ABC; D = 1 when the number of 1s is odd. Truth table and expression. / Simplify S = $A'B'C_i + A'BC_i' + AB'C_i' + ABC_i$ and draw it.
✎ **Asked in:** BUET M.Sc. · 15th NTRCA Computer Operation (2019)

Add ABC twice (idempotent) and pair: X = BC + AC + AB = **majority of A, B, C**. NAND: X = ((AB)′(BC)′(CA)′)′ → 3 two-input + 1 three-input NAND. NOR: X = (A + B)(B + C)(C + A) → 3 two-input + 1 three-input NOR.

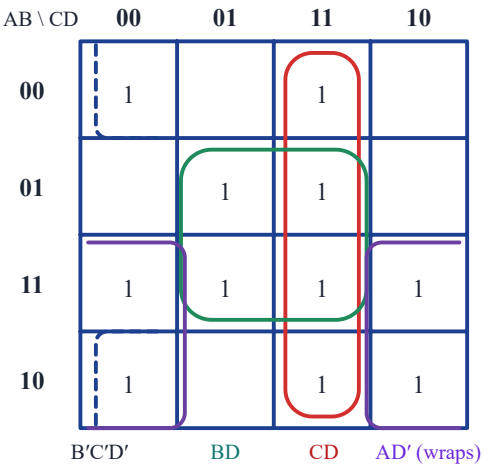
→ minterms 1, 2, 4, 7: **D = S = A ⊕ B ⊕ C** (the full-adder sum; no 2-level saving, use two XOR gates)

- What is a **minterm**? → a product containing every variable once (m5 = A′BC′D for 4 variables); "simplify F = ABC + ABD + ABC + AC" (bars lost in print) ✎ 15th NTRCA CSE
- Other printing: A′B′C′ + A′B′CD′ + A′BCD′ + A′BCD + AB′C′D + AB′CD + ABCD′ + ABCD = Σ(0, 1, 2, 6, 7, 9, 11, 14, 15) → **BC + A′B′C′ + A′B′D′ + AB′D** ✎ BOF AE(CSE) 2021
- Truth table of AB·(A + B)·C → 1 only for **ABC = 111** (it equals ABC) ✎ BB AD(ICT) 2025

10 K-map: the most-asked worked problem

Q27: Simplify F(A, B, C, D) = Σm(0, 3, 5, 7, 8, 10, 11, 12, 13, 14, 15) using a 4-variable K-map.

✎ **Asked in:** 46th BCS ICT · Combined Bank AME 2026 · Combined Bank AME/AE(IT) 2026(Jan) · Combined Bank SO(IT) 2026



1. Rows and columns in **Gray order** 00, 01, 11, 10, so neighbours differ in one bit.
2. Biggest groups first (8, 4, 2), powers of 2 only; edges **wrap**.
3. CD = column 11 (quad) · BD = middle square · AD′ = rows 11, 10 × columns 00, 10 (corner wrap) · m0 pairs with m8 → B′C′D′.
4. Every 1 covered, no group is redundant.

→ **F = CD + BD + AD′ + B′C′D′**

Exam trap: m0 and m8 are neighbours (AB = 00 and 10 differ in A only): the top and bottom rows touch.

Q28: Simplify F(A, B, C, D) = Σm(2, 8, 9, 11, 13, 15) using a K-map.

✎ **Asked in:** BCIC AP 2022 · BPDB AE(CSE) 2024 · Teletalk AM(IT) 2023

→ AD (9, 11, 13, 15) + AB′C′ (8, 9) + m2 alone → **F = AD + AB′C′ + A′B′CD′** (m2 has no 1 beside it)

Also asked: same method (every answer checked against its minterms)

- Σm(0, 2, 6, 7, 8, 10, 14, 15) → **B′D′ + BC** ✎ BPSC AP 2023 · BPSC AP 2026; the same 8 minterms written as terms → **B′D′ + BC** ✎ BOF AE(CSE) 2021
- 3-variable Σm(0, 3, 4, 5, 7) (A′B′C′ + AB′C′ + AB′C + A′BC + ABC) → **B′C′ + BC + AC** ✎ BPSC NE 2025 · JBL SO(ANE) 2020

- Σm(0, 2, 3, 7, 8, 10) → **B′D′ + A′CD** ✎ MRA SCO 2023 · Σ(0, 1, 7, 8, 13, 15) → **A′B′C′ + B′C′D′ + BCD + ABD** ✎ Parliament Secretariat SAE 2023
- Σ(2, 4, 6, 7, 10, 12, 14, 15) → **CD′ + BD′ + BC** ✎ 13th NTRCA CSE (2016) · Σ(3, 4, 5, 7, 9, 13, 14, 15) → **A′CD + A′BC′ + AC′D + ABC** ✎ Combined Bank AHE 2021
- f = A′BC′ + A′BC + A′B′C + ABC → **A′B + A′C + BC**, then gates: 1 NOT, 3 AND, 1 OR = **5**; with NOR only via POS ✎ 38th BCS (2018)
- A′B′C + AB′C + ABC′ + ABC → **B′C + AB** ✎ 46th BCS CSE · Σ(0, 2, 4, 6, 7) in x, y, z → **z′ + xy** ✎ NHA AP 2019 · A′B′C′ + A′B′C + A′BC′ + A′BC + ABC → **A′ + BC** ✎ MoFood Network/Website Manager 2025
- A′B′C′ + A′B + ABC′ + AC → **B + AC + A′C′** ✎ 16th NTRCA CSE (2019) · ABCD + AB′CD + A′BD + CD′ + A′B′CD → **C + A′BD** ✎ 18th NTRCA School-2 (2024)

> **Memory trick:** **"Gray it, wrap it, go big"**: Gray-code edges, wrap round, biggest power-of-2 groups.

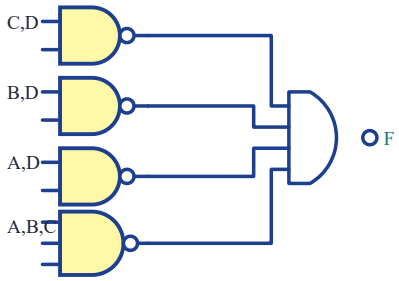
11 POS, don't cares and NAND-only circuits

> **Q29:** Simplify F(A, B, C, D) = Σ(3, 5, 7, 9, 11, 13, 14, 15) using a K-map and implement it with NAND gates only. Show the steps.
> ✎ **Asked in:** 47th BCS ICT · Combined Bank SO(AME) 2021

Groups: CD (3, 7, 11, 15) · BD (5, 7, 13, 15) · AD (9, 11, 13, 15) · ABC (14, 15) → **F = CD + BD + AD + ABC**.

SOP → NAND–NAND: double-invert and push one bar in: F = ((CD)′·(BD)′·(AD)′·(ABC)′)′. Every AND becomes NAND, the OR becomes NAND.

→ 3 two-input NAND + 1 three-input NAND + 1 four-input NAND = **5 NAND gates**



NAND–NAND = AND–OR

Other printing of the same paper: Σ(3, 5, 7, 9, 13, 14, 15) → **BD + A′CD + AC′D + ABC** ✎ 47th BCS ICT · NAND only for A′B′C′D′ + AB′CD′ + AB′C′D′ + A′CD + A′B′CD′ = Σ(0, 2, 3, 7, 8, 10) → **B′D′ + A′CD = ((B′D′)′(A′CD)′)′** ✎ MRA SCO 2023

> **Q30:** Minimize f(A, B, C, D) = Π(2, 3, 7, 12, 13, 14, 15) using a K-map. / Convert the canonical SOP Σ(1, 3, 5, 8, 11) to canonical POS.
> ✎ **Asked in:** 46th BCS ICT

A **Π list names the 0s**: group 0s for POS (0 → A, 1 → A′), 1s for SOP. Canonical POS = every index *not* in Σ.

→ SOP: **A′C′ + A′BD′ + AB′** · POS: **(A′ + B′)(A + B + C′)(A + C′ + D′)** · Σ(1, 3, 5, 8, 11) = **Π(0, 2, 4, 6, 7, 9, 10, 12, 13, 14, 15)**

Q31: F(A, B, C, D) = Σ(0, 2, 5, 7, 8, 9, 10, 13, 15): find the simplest form with a K-map.
✎ **Asked in:** BRPL AM(ICT) 2024

→ BD + B′D′ covers all but m9 → add AC′D (or AB′C′): **F = BD + B′D′ + AC′D**

Exam trap: A printed answer stopped at BD + B′D′: **m9 uncovered**.

Q32: 7-segment display: a segment is ON for BCD inputs 2, 3, 5, 6, 7, 8, 9. Write its SOP and simplify.
✎ **Asked in:** Combined Bank SO 2020

Inputs 10–15 never occur in BCD → **don't cares (X)**: use an X as 1 only when it enlarges a group.

→ with X: **A + C + BD** · without: A′C + A′BD + AB′C′

- a′b′c′d′ + a′b′c′d + a′bc′d + ab′cd′ → **a′b′c′ + a′c′d + ab′cd′** ✎ NWPGCL JAM 2020 · ACD + AB + D′ + ACD′ → **D′ + AB + AC** ✎ BPSC ADA 2022
- Σ(3, 5, 7, 9, 10, 13, 14, 15) in w, x, y, z → **xz + w′yz + wy′z + wyz′** ✎ DUET AP 2025 · at least two of three inputs = 1 → **AB + BC + CA** ✎ Jamuna Oil JO(IT & MIS) 2025
- Garbled function, read as A′B′C + A′BC + A′BC′D + ABC′D + ABCD′ + AB′C′D′ → **A′C + BC′D + BCD′ + AB′C′D′** ✎ Combined Bank SO(IT) 2022
- Cited only (stem or figure not in the source): easy K-map with don't cares ✎ BUET M.Sc. October 2017 · canonical SOP via K-map ✎ BUET M.Sc. April 2017 · "minimise the given function" ✎ Teletalk AM(IT) 2023 · Petrobangla AM(IT) 2022 · BUET M.Sc. May 2016 · Hydrocarbon Unit AD(ICT) 2025

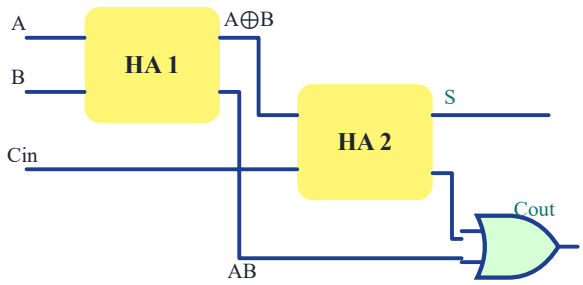
12 Half adder, full adder, n-bit adder

Q33: Half and full adder: build a full adder from two half adders and an OR gate; truth table and output functions.
✎ **Asked in:** MoFood Network/Website Manager 2025 · Combined Bank SO(IT) 2020 · BPSC SAE 2021 · BPSC Lecturer 2017 · 15th NTRCA Computer Operation (2019) · 16th NTRCA CSE (2019) · 47th BCS ICT · Cadet College Lecturer(ICT) 2025 · 15th NTRCA CSE

Half adder: S = A ⊕ B, C = AB. **Full adder** (A, B, C_{in}) — "sum is XOR, carry is majority":

$S = A \oplus B \oplus C_{in}$

$C_{out} = AB + C_{in}(A \oplus B)$ = $AB + BC_{in} + AC_{in}$



2 HA + 1 OR = 1 FA

A B C_{in}	S	C_{out}	A B C_{in}	S	C_{out}
0 0 0	0	0	1 0 0	1	0
0 0 1	1	0	1 0 1	0	1
0 1 0	1	0	1 1 0	0	1
0 1 1	0	1	1 1 1	1	1

Also asked: half adder circuit and truth table ✎ 15th NTRCA CSE · full adder truth table from a given 2-bit + carry circuit ✎ Sonali Bank ADBA 2024 · MCQ: carry-out of a full adder with inputs x, y, z → **xy + xz + yz** ✎ 49th BCS ICT (2025)

Q34: Design a 4-bit binary adder using 1-bit full adders. / Design a 4-bit subtractor using a 4-bit adder.

✎ **Asked in:** 36th BCS (2016) · BTCL JAM 2024

Ripple-carry adder: chain 4 FAs, C_{out} of stage i → C_{in} of stage i+1, $C_0 = 0$. **Subtractor:** $A - B = A + B' + 1$ → put an XOR on every B input ($B \oplus M$) and feed **M = 1** into C_0. With M = 0 the same circuit adds (adder–subtractor).

Q35: Design a combinational circuit that gives the 2's complement of a 4-bit binary number.

✎ **Asked in:** 47th BCS CSE

"Copy up to the first 1, flip the rest": **Y0 = A0**, Y1 = A1 ⊕ A0, Y2 = A2 ⊕ (A1 + A0), Y3 = A3 ⊕ (A2 + A1 + A0) → 3 XOR + 2 OR (checked for all 16 inputs). Or: 4 NOT into a 4-bit adder with B = 0001.

Q36: Main difference between combinational and sequential circuits, with diagrams.

✎ **Asked in:** WZPDCL AE(CSE) 2019 · Fisheries & Livestock Instructor(CS) 2023 · 47th BCS ICT

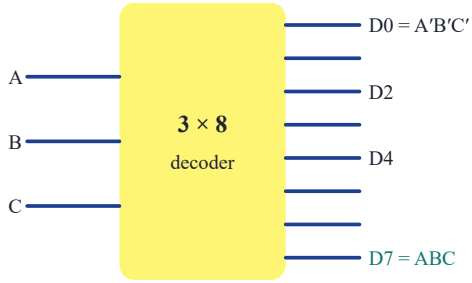
Combinational: output = f(present inputs) only; no memory, no feedback, no clock (adder, MUX, decoder). **Sequential:** output = f(inputs, **stored state**); **memory + feedback**, usually clocked (register, counter, FSM).

13 Decoders, encoders and multiplexers

Q37: What is a decoder? Draw the truth table and circuit of a 3-to-8 line decoder.

✎ **Asked in:** 13th NTRCA CSE (2016) · 17th NTRCA CSE (2023)

A **decoder** turns an n-bit code into one of 2^n outputs: output $D_i = 1$ only for input i, so each output is one **minterm**. 3-to-8: D0 = A′B′C′, D1 = A′B′C … D7 = ABC → 3 NOT + 8 three-input AND (plus an enable on every AND).



Q38: Design a 5 × 32 decoder with four 3 × 8 decoders and one 2 × 4 decoder (block diagram).

✎ **Asked in:** 45th BCS ICT (2024)

The 2 × 4 decodes the two **MSBs** (A4 A3) and its 4 outputs drive the **enables** of the four 3 × 8s. A2 A1 A0 go to all four in parallel. Only the enabled 3 × 8 drives a line: outputs D0–D7, D8–D15, D16–D23, D24–D31.

Q39: Implement F(A, B, C, D) = Σ(0, 5, 6, 8, 13, 14) using a MUX.

✎ **Asked in:** 45th BCS ICT (2024)

8 : 1 MUX, selects A B C, data from D. For each ABC pair (m2k, m2k+1): both 1 → 1, neither → 0, only even → D′, only odd → D.

→ I0…I7 = **D′, 0, D, D′, D′, 0, D, D′**

Q40: What is a multiplexer? Design a 4 : 1 MUX (logic diagram) and give the job of its two selection lines; write its expression in SOP form.

✎ **Asked in:** NWPGCL JAM 2020 · Titas Gas AE(CSE) 2024 · BTCL AE(CSE) 2017 · 16th NTRCA ICT

A **MUX** sends one of 2^n inputs to one output; the n **select lines** choose which. **Y = S1′S0′I0 + S1′S0 I1 + S1 S0′ I2 + S1 S0 I3** → 2 NOT + 4 three-input AND + 1 four-input OR.

- Half adder from a 2 : 4 decoder + OR → **S = D1 + D2** (OR), **C = D3** ✎ BTCL JAM 2024 · XOR from a 2-to-4 decoder + NOR → **A⊕B = (D0 + D3)′** ✎ BUET M.Sc. April 2024
- 2 × 4 decoder from basic gates → 2 NOT + 4 AND (D0 = A′B′ … D3 = AB) ✎ BUET M.Sc. May 2016 · 8-to-3 encoder → **A2 = D4+D5+D6+D7, A1 = D2+D3+D6+D7, A0 = D1+D3+D5+D7** ✎ Cadet College Lecturer(ICT) 2025
- 7-segment decoder for a 2-bit input (digits 0–3) → **a = d = B1 + B0′, b = 1, c = B1′ + B0, e = B0′, f = B1′B0′, g = B1** ✎ Rupali Bank ANE 2021

Also asked: MCQ: the basic difference lies in → **feedback (memory) used** ✎ 49th BCS ICT (2025) · "a lamp that is OFF in sunlight and ON without it" → light sensor S (1 = light), **Lamp = S′** (one NOT driving a relay) ✎ DPDC AM(ICT) 2025

Memory trick: **"Decoder makes minterms, MUX picks one"**: any n-variable function = a decoder + OR, or a 2^{n-1} : 1 MUX with the last variable on the data lines.

14 Latch vs flip-flop: the single most-asked question

Q41: What is the difference between a latch and a flip-flop?

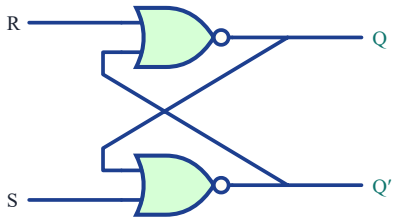
✎ **Asked in:** 36th BCS (2016) · BB AME 2017 · BPSC AP 2021 · BWDB AP 2018 · SPCBL AME 2021 · BCPCL JI 2019

	latch	flip-flop
changes	**level-triggered**: follows input while enable = 1	**edge-triggered**: only at the clock edge
clock	none, or an enable (gated latch)	clock input required
timing	transparent: glitches pass	one change per edge

MCQs: main difference → **FF edge-triggered, latch level-triggered** ✎ 49th BCS CSE (2025) · BUET IICT April 2021 · **Register vs latch:** a register is n flip-flops sharing a clock; a latch is one level-sensitive bit ✎ WZPDCL AE(CSE) 2019

Q42: What is a flip-flop? Describe the structure of an S-R flip-flop with its truth table and properties.

✎ **Asked in:** BPSC ANE 2020 · 15th NTRCA CSE



cross-coupled NOR latch

A **flip-flop** is a 1-bit memory with two stable states (Q, Q′). S-R from two cross-coupled NOR gates: S = 1 sets, R = 1 resets, S = R = 0 holds, **S = R = 1 forbidden** (Q = Q′ = 0).

flip-flop	characteristic equation	input rule	note
SR	$Q^+ = S + R'Q$ (SR = 0)	10 set, 01 reset, 00 hold	11 forbidden
JK	$\mathbf{Q^+ = JQ' + K'Q}$	as SR, but 11 → toggle	no forbidden state
D	$Q^+ = D$	output copies D at the edge	registers
T	$Q^+ = T \oplus Q$	1 → toggle, 0 → hold	counters

Q43: Describe a J-K flip-flop with figure and truth table. / Show the differences between clocked J-K and clocked D flip-flops.

✎ **Asked in:** 18th NTRCA School-2 (2024) · 45th BCS CSE (2024)

JK: two inputs, four actions (hold, reset, set, **toggle**). D: one input, no hold/toggle by itself, no invalid state; D = JK with K = J′.

Q44: Explain the race-around condition in flip-flops and how it is resolved.

✎ **Asked in:** 46th BCS ICT

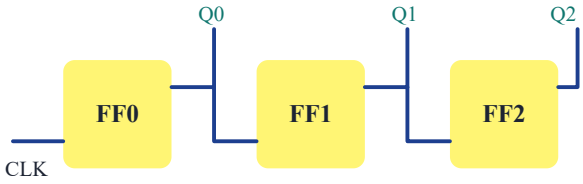
With J = K = 1 and a clock pulse **wider than the propagation delay**, Q toggles again and again during one pulse, ending unpredictably. Fix: **master–slave JK** or an **edge-triggered** flip-flop (or $t_{pulse} < t_{pd}$).

15 Ripple counters, MOD-N and state machines

Q45: What is a counter (ripple counter)? Describe a 3-bit asynchronous up (ripple) counter. / Design an asynchronous counter with any flip-flop.

✎ **Asked in:** 13th NTRCA CSE (2016) · 15th NTRCA CSE · 16th NTRCA CSE (2019) · BAERA Engineer(CSE) November 2021

A **counter** is a register that steps through a fixed sequence on clock pulses. **Ripple (asynchronous)**: only FF0 gets the clock; each Q drives the clock of the next FF, so the change ripples. T (or JK) flip-flops, T = 1, falling edge.



T = 1 on each · Q of one = clock of next
output Q2 Q1 Q0 counts 0 → 7

→ 3 FFs count 000 → 001 → … → 111 → 000: **$2^3 = 8$ states** (MOD-8); delay = 3 t_{pd}

> **Q46:** Draw a MOD-12 ripple counter and explain it. / Draw a MOD-12 up counter using only T flip-flops.
> ✎ **Asked in:** 46th BCS CSE · BUET M.Sc. April 2017

$2^3 < 12 \le 2^4$ → **4 flip-flops**. Count 0–11; the first state with Q3 = Q2 = 1 is **12 = 1100** → NAND(Q3, Q2) drives every CLR, so 12 lasts a moment and the counter returns to 0000.

> **Q47:** Draw the block diagram of a MOD-6 binary counter.
> ✎ **Asked in:** NWPGCL JAM 2020

3 flip-flops ($2^2 < 6 \le 2^3$); first state with Q2 = Q1 = 1 is **6 = 110** → NAND(Q2, Q1) → CLR. Sequence 000 … 101, then back to 000.

> **Q48:** A traffic light cycles GREEN 70 s → YELLOW 5 s → RED 75 s, driven only by a 5 s clock. What is the minimum number of flip-flops for the state machine?
> ✎ **Asked in:** Petrobangla AM(CSE) 2024

Tick = 5 s; cycle 150 s → 150 / 5 = **30 states** (14 green, 1 yellow, 15 red). FFs = $\lceil \log_2 30 \rceil$: $2^4 = 16 < 30 \le 32 = 2^5$.

→ **5 flip-flops** (the source printed no answer)

- Convert a 100 MHz clock to 50 MHz and 25 MHz with D flip-flops → D = Q′ divides by 2 → **two cascaded D FFs**: 50 MHz after the first, 25 MHz after the second ✎ BUET M.Sc. October 2017
- States in a 4-bit BCD ripple counter → **10** ✎ 49th BCS ICT (2025) · Johnson counter → **the inverted output of the last FF feeds the first** (n FFs, 2n states) ✎ BUET IICT April 2021
- Convert J-K to T → tie **J = K = T**; MCQ answer: **no extra gate is required** ✎ 36th BCS (2016) · BUET IICT April 2021
- Advantage of JK over SR → **no forbidden state** · asynchronous input of a D flip-flop with D, CLK, CLR → **CLR** ✎ BUET IICT April 2021
- JK with J = X ⊕ Q, K = X′, output A = Q′ → $Q^+ = (X \oplus Q)Q' + XQ = X$, so **A(t+1) = X′(t)** ✎ BUET M.Sc. April 2021

16 Everything on one sheet

Gates

AND all 1 · OR any 1 · XOR odd 1s (anticoincidence) · XNOR equal (coincidence) · NAND, NOR = **universal**. NAND: NOT 1, AND 2, OR 3 · NOR: NOT 1, OR 2, AND 3 · XOR = 4 NAND.

Boolean laws

$A + 1 = 1$ · $A \cdot 0 = 0$ · $A + AB = A$ · $\mathbf{A + A'B = A + B}$ · $A + BC = (A + B)(A + C)$ · consensus $AB + A'C + BC = AB + A'C$ · De Morgan: break the bar, change the sign.

K-map

Gray order 00 01 11 10 · groups of 1, 2, 4, 8, 16 · edges and corners wrap · largest groups first · don't care X = 1 only if it helps · Π lists the 0s · canonical POS = indices not in Σ.

Numbers

hex = 4 bits, octal = 3 bits, group from the point · integer: divide, read up · fraction: multiply, read down · 2's complement = invert + 1 = $2^n - x$ · 8-bit range −128…+127 · $A - B = A + B' + 1$, drop the carry, MSB 1 → negative · BCD: +0110 when a digit > 9 · ASCII 7 bits: A 65, a 97, 0 48.

Combinational

HA: $S = A \oplus B$, $C = AB$ · FA: $S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$ · FA = 2 HA + OR · subtractor = adder + XOR on B, $C_0 = 1$ · n-to-2^n decoder outputs = minterms · MUX 4:1: $Y = \Sigma$ S-minterm·I · n-variable function = 2^{n-1} : 1 MUX.

Sequential

FF	Q^+
SR	S + R′Q (11 invalid)
JK	JQ′ + K′Q (11 toggles)
D	D
T	T ⊕ Q

Latch = level, FF = edge · race-around → master–slave / edge · JK → T: J = K = T · flip-flops for N states = $\mathbf{\lceil \log_2 N \rceil}$ · MOD-N: NAND the 1-bits of N into CLR · Johnson: Q′ of last → first (2n states) · BCD counter: 10 states.

> **Traps seen in printed solutions:** $(E)_{16} = \mathbf{14}$ · "$(108)_8$" has no meaning (8 is not octal) · a K-map answer that leaves a 1 (m9) uncovered · a truth table with a wrong middle column · 2's complement "of +55" = −55 = 11001001 · $AB + (A' + B)C = AB + A'C$, not $AB + C$ · copy base subscripts exactly $((9D \cdot AB6)_{16} + (306 \cdot 51)_{10})$.

Real MCQs to remember

- MCQ: K-map with AB on rows, CD on columns: row 1, column 3 → **A′B′CD** ✎ BUET IICT April 2021
- 4/1 MUXes needed for a 16/1 MUX → **5** (4 + 1) ✎ BUET IICT April 2021 · in a priority encoder → **lower-priority bits are don't care** ✎ BUET IICT April 2021
- Priority-encoder table (D1 wins, then D0, D3, D2) → order **1, 0, 3, 2** ✎ BUET M.Sc. April 2021

- 0001 into a 7-segment decoder → 0110000 common-cathode and 1001111 common-anode → **both true** ✎ BUET IICT April 2021
- 4 : 1 MUX with I0 = I1 = Z′, I2 = 0, I3 = 1, selects X Y → **F = Σ(0, 2, 6, 7)** ✎ BUET M.Sc. April 2021 · a PLA is mainly used for → **custom combinational circuits** ✎ 49th BCS ICT (2025)

17 Practice under exam conditions

1. Which one is a universal logic gate? ✎ BB AD(ICT) 2025

(a) NAND (b) AND
(c) OR (d) NOT

2. The main difference between a latch and a flip-flop: ✎ 49th BCS CSE (2025)

(a) FF level, latch edge (b) FF edge, latch level
(c) FF can't store (d) latch stores more

3. A programmable logic array is mainly used for: ✎ 49th BCS ICT (2025)

(a) custom sequential (b) memory
(c) custom combinational (d) fixed logic

4. In modulo-2 arithmetic, ____ give the same result: ✎ 49th BCS ICT (2025)

(a) add & divide (b) add & multiply
(c) multiply & divide (d) subtract & add

5. Priority order of an encoder with D1→01, D0→00, D3→11, D2→10? ✎ BUET M.Sc. April 2021

(a) 1, 0, 3, 2 (b) 1, 2, 3, 4
(c) 4, 3, 2, 1 (d) 1, 2, 4, 3

6. 4:1 MUX, I0 = I1 = Z′, I2 = 0, I3 = 1, S1S0 = XY. F = ✎ BUET M.Sc. April 2021

(a) Σ(0,1,2,3) (b) Σ(0,2,6,7)
(c) Σ(2,3) (d) Σ(0,1)

7. 0001 applied to a seven-segment decoder gives: ✎ BUET IICT April 2021

(a) 0110000 (CC) (b) 1001111 (CA)
(c) both a and b (d) none

8. JK: J = X⊕Q, K = X′, A = Q′. A(t+1) = ✎ BUET M.Sc. April 2021

(a) 1 (b) X + A
(c) 0 (d) X′

9. 4/1 MUXes needed to build a 16/1 MUX: ✎ BUET IICT April 2021

(a) 3 (b) 4
(c) 5 (d) 6

10. Number of states in a 4-bit BCD ripple counter: ✎ 49th BCS ICT (2025)

(a) 4 (b) 10
(c) 16 (d) 5

Answer key → 1-a · 2-b · 3-c · 4-d · 5-a · 6-b · 7-c · 8-d · 9-c · 10-b

Written: Simplify F(A, B, C, D) = Σm(0, 3, 5, 7, 8, 10, 11, 12, 13, 14, 15) using a K-map and draw the circuit with basic gates. [10]

✎ **Asked in:** 46th BCS ICT · Combined Bank AME 2026 · Combined Bank AME/AE(IT) 2026(Jan) · Combined Bank SO(IT) 2026

1. **[2]** 4 × 4 K-map, AB rows and CD columns in Gray order, eleven 1s placed.
2. **[3]** Groups CD · BD · AD′ (corner wrap) · B′C′D′ (m0 + m8); each covers a 1 no other group does.
3. **[2]** **F = CD + BD + AD′ + B′C′D′**; check m12 = 1100 → AD′ = 1.
4. **[3]** 3 NOT → 3 two-input AND + 1 three-input AND → 4-input OR = **8 gates**.

Before the exam ✓ Gray labels, tick each 1 · draw NOT/AND/OR to prove "universal" · fill every column · 8-bit 2's complement · $\lceil \log_2 N \rceil$ FFs.